



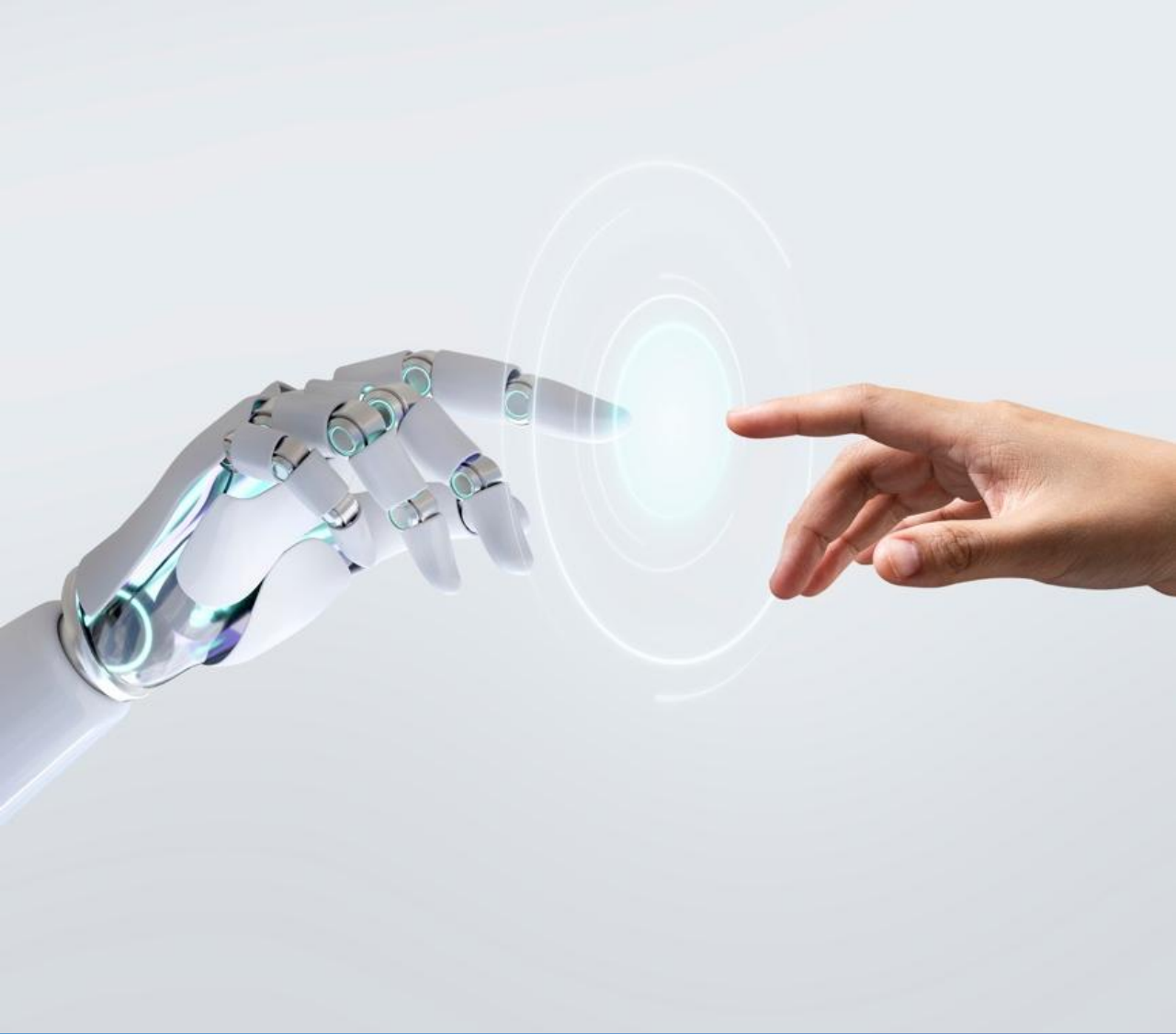
Ahmed Irshad Hussain

sardarahmed.irshad2@gmail.com

© COPYRIGHT ALL RIGHTS RESERVED

CLOUD DATA ENGINEERING:
MICROSOFT FABRIC
Live API Data Engineering Pipeline

By: Ahmed Irshad Hussain



Data Engineering: Microsoft Fabric
Live API Data Engineering Pipeline

API Ingestion

This is one of the MOST important skills for modern data engineers.

Because in industry:

data rarely comes from CSVs,

systems continuously generate live data through:

- APIs
- applications
- web services
- IoT devices
- streaming systems

WHAT IS AN API?

API = Application Programming Interface

Think of it as:

A way for systems to communicate and exchange data

REAL-WORLD EXAMPLES

Companies pull data from APIs like:

- weather APIs
- payment APIs
- e-commerce APIs
- social media APIs
- stock market APIs
- healthcare systems

WHAT DATA ENGINEERS DO

Data Engineers:

- Call APIs
- Extract JSON data
- Store raw data in Bronze
- Clean in Silver
- Aggregate in Gold
- Visualize in Power BI

This is REAL enterprise ingestion.

Current:

- CSV → Bronze → Silver → Gold

After this:

- API → Bronze → Silver → Gold

Now my pipeline will become:

- dynamic and live

WHY APIs MATTER

Because APIs teach:

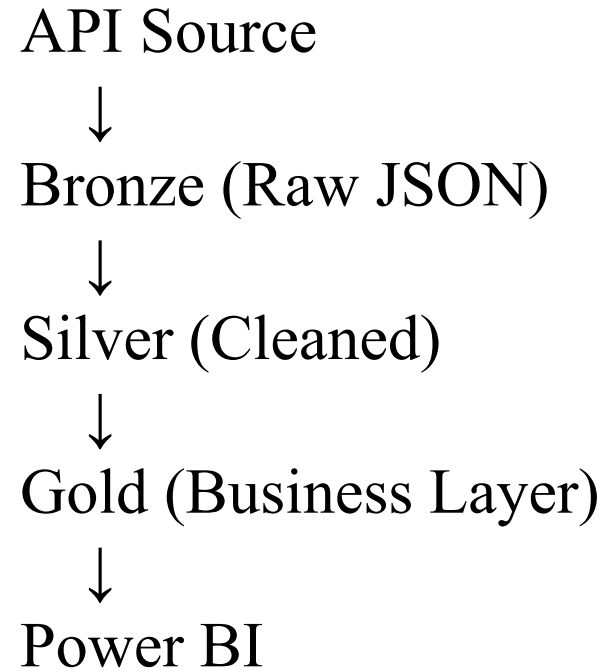
- live ingestion
- automation
- JSON handling
- schema evolution
- production pipelines

These are CORE modern data engineering skills.

EXPECTED KEY LEARNINGS:

- REST APIs
- JSON Parsing
- API Authentication
- Dynamic Bronze Ingestion

MY PIPELINE WILL BECOME



Microsoft Fabric

Weather API

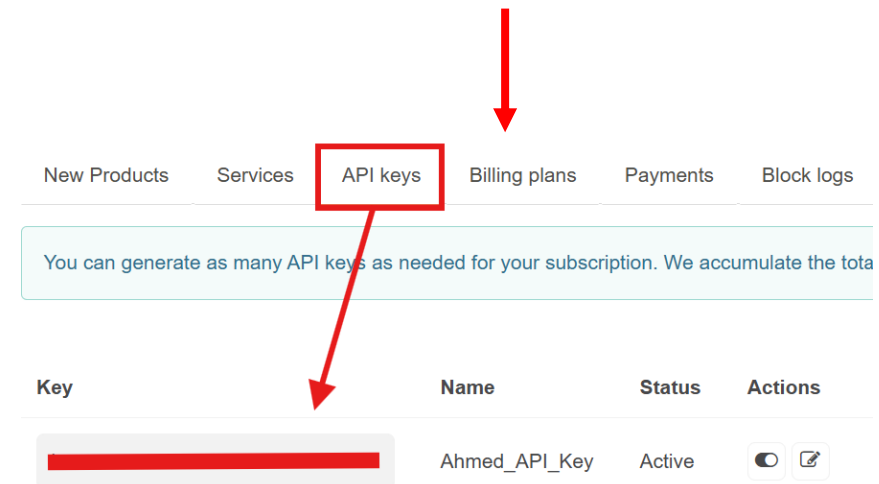
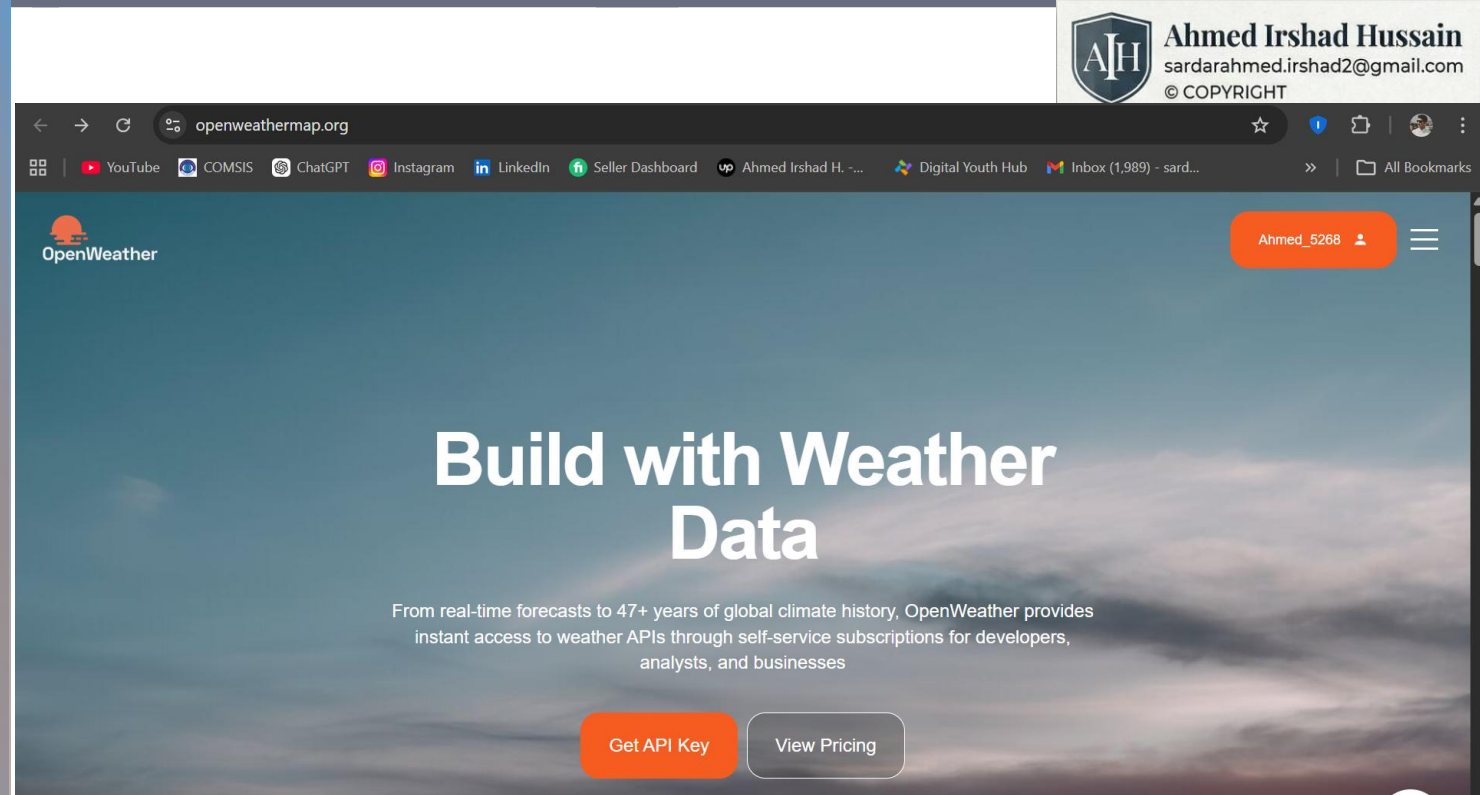
I'll use weather API for this project

STEP 1: Go to “Open Weather”

- Create your account first.
- After that click your user's name tab!
(You can see it at top right corner)

STEP 2: Open API Tab:

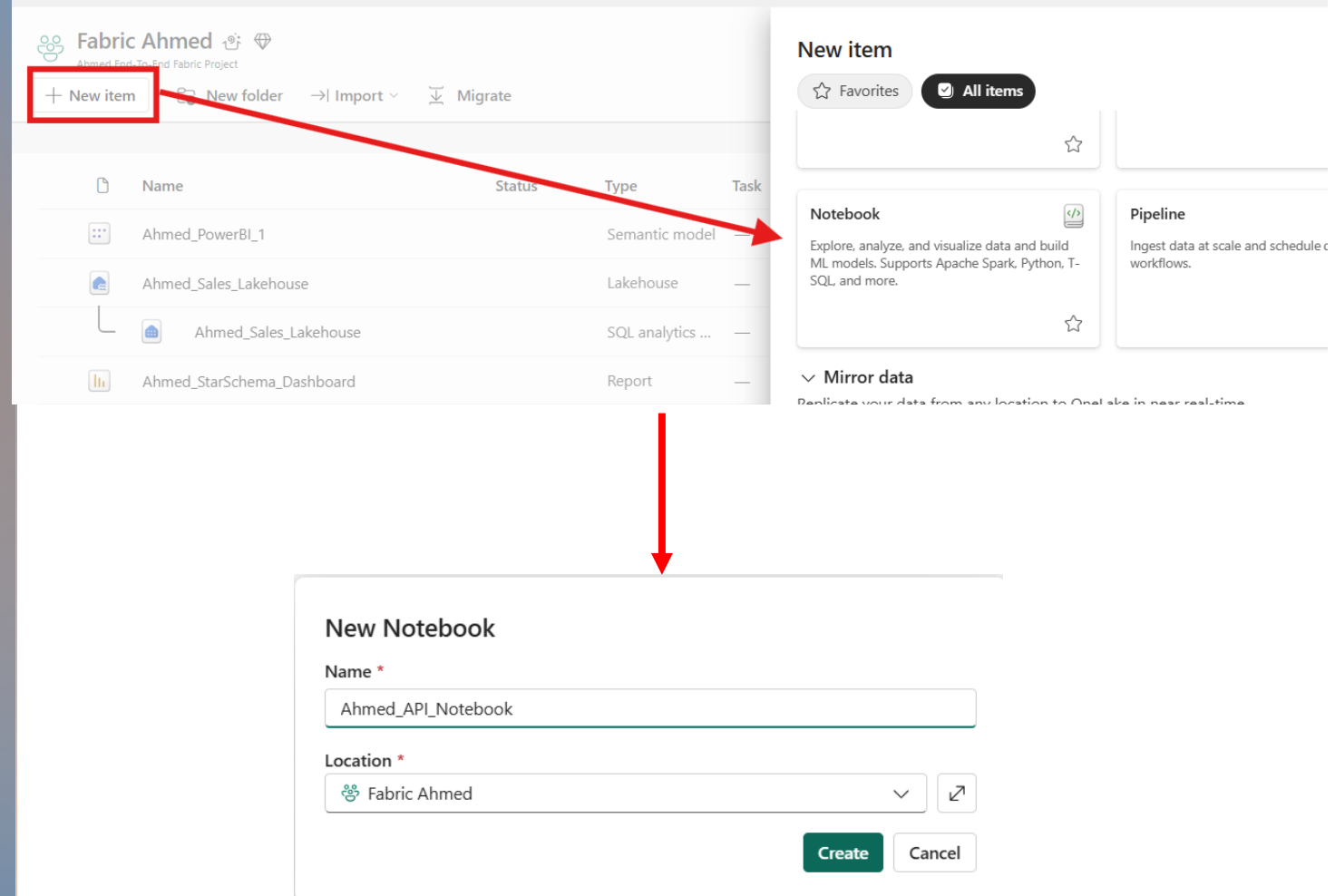
- Click API keys
- And generate your own API key.



Microsoft Fabric

STEP 3: Creating New Notebook

- I'll create new notebook for this project.
As my old notebook was based on CSV files.
- Naming it “Ahmed API Notebook”
You all can choose what you like!



The screenshot shows the Microsoft Fabric interface for a project named 'Fabric Ahmed'. The 'New item' button is highlighted with a red box, and a red arrow points from it to the 'New Notebook' dialog. The dialog shows the name 'Ahmed_API_Notebook' and the location 'Fabric Ahmed'.

Fabric Ahmed
Ahmed End-To-End Fabric Project

+ New item New folder → Import Migrate

	Name	Status	Type	Task
	Ahmed_PowerBI_1		Semantic model	—
	Ahmed_Sales_Lakehouse		Lakehouse	—
	Ahmed_Sales_Lakehouse		SQL analytics ...	—
	Ahmed_StarSchema_Dashboard		Report	—

New item

☆ Favorites **☑ All items**

Notebook
Explore, analyze, and visualize data and build ML models. Supports Apache Spark, Python, T-SQL, and more.

Pipeline
Ingest data at scale and schedule workflows.

▼ **Mirror data**
Replicate your data from any location to OneLake in near real-time.

New Notebook

Name *
Ahmed_API_Notebook

Location *
Fabric Ahmed

Create **Cancel**

Microsoft Fabric

STEP 4: API Ingestion

- Inside code, write first API call!



```
1 import requests
2 import json
3
4 API_KEY = "1379c54556414250c53125c68214"
5
6 url = f"https://api.openweathermap.org/data/2.5/weather?q=London&appid={API_KEY}&units=metric"
7
8 response = requests.get(url)
9
10 print("Status Code:", response.status_code)
11
12 data = response.json()
13
14 print(json.dumps(data, indent=4))
15
```

Requests = Used to call APIs over internet
JSON= Used to format and work with
JSON data

API_Key will store my key for authentication credential.
Without it, API rejects request

These 2 lines, will send HTTP request.
And as well as, will check response status.

API returns raw JSON, it'll convert into python dictionary
format

This line explanation is in next slide!

This builds the API endpoint
dynamically.
q= London, means CITY
appid = API_Key, for authentication
Units = metric, means Celsius Temp.

[1] ✓ 11 sec - Session ready in 11 sec 178 ms. Command executed in 533 ms by Ahmed Fabric on 5/11/2026, 12:07:50 AM

```
... Status Code: 200
{
  "coord": {
    "lon": -0.1257,
    "lat": 51.5085
  },
  "weather": [
    {
      "id": 804,
      "main": "Clouds",
      "description": "overcast clouds",
      "icon": "04d"
    }
  ],
  "base": "stations",
  "main": {
    "temp": 11.68,
    "feels_like": 10.39,
    "temp_min": 10.95,
    "temp_max": 12.38,
    "pressure": 1014,
    "humidity": 57,
    "sea_level": 1014,
    "grnd_level": 1009
  },
  "visibility": 10000,
  "wind": {
    "speed": 4.63,
    "deg": 40
  },
  "clouds": {
    "all": 100
  },
  "dt": 1778439324,
  "sys": {
    "type": 2,
    "id": 268730,
    "country": "GB",
    "sunrise": 1778386604,
    "sunset": 1778441828
  },
  "timezone": 3600,
  "id": 2643743,
  "name": "London",
  "cod": 200
}
```

As, status code = 200
It means, response successfully generated

Last line was for “Pretty Print JSON”

Example:

Without it:

```
{'temp':20,'humidity':60}
```

With It:

```
{
  "temp": 20,
  "humidity": 60
}
```

Much Cleaner, as you can see in success status response.

Microsoft Fabric

STEP 5: Connecting Notebook to lakehouse

- Click, Add new items.
- Select “One Lake Catalog”.
- Then add the lakehouse.

STEP 6: Save into Bronze

Currently:

- data exists only temporarily in memory

I'll now persist/store it into:

- Delta Table, inside my Lakehouse.



OneLake catalog

Discover data from your org and beyond and use it to create reports

All Endorsed in your org My data Favorites

No data sources added

Add data items

	Name	Owner	Refreshed	Location
☐	Ahmed_Sales_Lakehouse	Ahmed Fabric	—	Fabric Ahmed
☐	Ahmed_Sales_Lakehouse	Ahmed Fabric	—	Fabric Ahmed

```
1 import requests
2 import json
3
4 # API Key
5 API_KEY = "ba979c51e5a6d44c250e53ab25c603bb"
6
7 # API URL
8 url = f"https://api.openweathermap.org/data/2.5/weather?q=London&appid={API_KEY}&units=metric"
9
10 # Fetch Data
11 response = requests.get(url)
12
13 # Convert response to JSON
14 data = response.json()
15
16 # Convert JSON dictionary to string
17 json_data = json.dumps(data)
18
19 # Create Spark DataFrame
20 df = spark.read.json(sc.parallelize([json_data]))
21
22 # Show DataFrame
23 df.show()
24
25 # Save Bronze Layer
26 df.write.format("delta") \
27     .mode("overwrite") \
28     .saveAsTable("bronze_weather")
```

[2] ✓ 24 sec - Command executed in 24 sec 316 ms by Ahmed Fabric on 5/11/2026, 12:34:37 AM

> Spark jobs (11 of 11 succeeded) Resources



Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com
© COPYRIGHT

VERIFYING



Ahmed Irshad Hussain
sardarahmed.irshad2@gmail.com
© COPYRIGHT

Fabric

Ahmed_API_Notebook

Ahmed_Sales_Lakehouse

Search

Trials activated: 26 days left

10

Home

Materialized lake views

Lakehouse

Share

Get data

New semantic model

Open notebook

Add to data agent

Manage OneLake security

Update all variables

Explorer

Add lakehouses

Ahmed_Sales_Lakehouse

Tables

dbo

bronze_sales

bronze_weather

dim_date

dim_product

fact_sales

gold_sales

silver_sales

Files

Bronze

bronze_weather

Table view

	ABC base	{ } clouds	12L cod	{ } coord	12L dt	12L id	{ } main	ABC name	{ } sys	12L timezone	12L visibility	{ } weather	{ } wind
1	stations	{ "all": 100 }	200	{ "lat": 51.5085, "l...	1778441172	2643743	{ "feels_like": 9.7...	London	{ "country": "GB",...	3600	10000	[{ "description": ...	{ "deg": 50, "sp

Microsoft Fabric

SILVER LAYER

Currently Bronze contains:

- nested JSON
- raw structure
- unnecessary metadata

Silver layer will:

- flatten JSON
- clean columns
- select useful fields
- create structured analytics-ready table

STEP 7: Create the Silver Layer

- Click, Add new items.
- Select “One Lake Catalog”.
- Then add the lakehouse.

▶ ▼

```
1  from pyspark.sql.functions import col
2
3  # Read Bronze Layer
4  df = spark.table("bronze_weather")
5
6  # Flatten & Clean Data
7  silver_df = df.select(
8      col("name").alias("city"),
9      col("main.temp").alias("temperature"),
10     col("main.humidity").alias("humidity"),
11     col("weather")[0]["main"].alias("weather_condition"),
12     col("wind.speed").alias("wind_speed"),
13     col("sys.country").alias("country")
14 )
15
16 # Show Silver Data
17 silver_df.show()
18
19 # Save Silver Layer
20 silver_df.write.format("delta") \
21     .mode("overwrite") \
22     .saveAsTable("silver_weather")
```

[3] ✓ 37 sec - Command executed in 37 sec 625 ms by Ahmed Fabric on 5/15/2026, 5:09:39 PM

What this does?

Bronze (Raw JSON)
Messy nested structure:

	ABC base	{ } clouds	12L cod	{ } coord	12L dt	12L id	{ } main	ABC name
1	stations	{ "all": 75 }	200	{ "lat": 51.5085, "...	1778845875	2643743	{ "feels_like": 10....	London

Silver Structured:

	ABC city	12 temperature	12L humidity	ABC weather_condition	12 wind_speed	ABC country
1	London	12.12	58	Clouds	5.66	GB

Microsoft Fabric

GOLD LAYER

We'll create:

- temperature categories
- weather status
- analytics-ready fields

Example:

city	temperature	temp_category
London	12	Cold



```

1  from pyspark.sql.functions import when, col
2
3  # Read Silver Layer
4  df = spark.table("silver_weather")
5
6  # Create Gold Layer
7  gold_df = df.withColumn(
8      "temp_category",
9      when(col("temperature") < 10, "Cold")
10     .when(col("temperature") < 20, "Moderate")
11     .otherwise("Hot")
12 )
13
14 # Show Gold Data
15 gold_df.show()
16
17 # Save Gold Layer
18 gold_df.write.format("delta") \
19     .mode("overwrite") \
20     .saveAsTable("gold_weather")

```

[4] ✓ 14 sec - Command executed in 14 sec 71 ms by Ahmed Fabric on 5/15/2026, 5:26:21 PM



```

... +-----+-----+-----+-----+-----+-----+-----+
| city|temperature|humidity|weather_condition|wind_speed|country|temp_category|
+-----+-----+-----+-----+-----+-----+-----+
|London|    12.12|    58|          Clouds|    5.66|    GB|    Moderate|
+-----+-----+-----+-----+-----+-----+-----+

```

WHAT I JUST BUILT

I now have:

- LIVE API → Bronze → Silver → Gold , pipeline.

What's Next?

NOW I'll evolve this into: ENTERPRISE API ENGINEERING

Because currently: I fetch one snapshot only

Right now: London → current weather gets overwritten every run.

Instead: I'll APPEND records continuously.

Meaning:

city	temp	ingestion_time
London	12	5:00 PM
London	13	6:00 PM
London	11	7:00 PM

NOW: trends become possible , analytics become meaningful and time-series engineering begins

WHAT I'LL IMPLEMENT NEXT

- ingestion timestamp
- append mode
- incremental API ingestion
- partitioning by date
- historical analytics
- automated scheduling

THIS becomes:

production-grade API pipeline

Now I'll move from “Current weather Snapshot” to “Historical weather pipeline”.

This is incremental load, but now for APIs!

Instead of overwriting:

London → 12°C

Every run will keep history

city	temp	ingestion_time
London	12.1	5:00 PM
London	13.3	6:00 PM
London	11.8	7:00 PM

After this, I can build,

Hourly trends

Time-series analytics

Growth analysis

Historical dashboards

Microsoft Fabric

Step 1 – Add Ingestion Timestamp

Run

Step 2 – Changing Bronze Write Mode:

Replacing: .mode(“overwrite”)

With: .mode(“append”)

```

1  import requests
2  import json
3  from pyspark.sql.functions import current_timestamp
4
5  # API Key
6  API_KEY = "ba979c51e5a6d44c250e53ab25c603bb"
7
8  # API URL
9  url = f"https://api.openweathermap.org/data/2.5/weather?q=London&appid={API_KEY}&units=metric"
10
11 # Fetch API data
12 response = requests.get(url)
13
14 # Convert to JSON
15 data = response.json()
16
17 # Create Spark DataFrame
18 json_data = json.dumps(data)
19
20 df = spark.read.json(sc.parallelize([json_data]))
21
22 # Add ingestion timestamp
23 df = df.withColumn(
24     "ingestion_time",
25     current_timestamp()
26 )
27
28 # Show result
29 df.show()

```

[6] ✓ 7 sec - Command executed in 7 sec 962 ms by Ahmed Fabric on 5/23/2026, 2:25:22 PM

What's changed,

Earlier:

London | 12°C

Now:

London | 12°C | 2026-05-15 17:40:23

This Timestamp becomes my:

watermark / historical tracking column



```

31 df.write.format("delta") \
32     .mode("append") \
33     .saveAsTable("bronze_weather_history")

```

✓ 9 sec - Command executed in 8 sec 401 ms by Ahmed Fabric on 5/23/2026, 2:31:10 PM

Why Append?

Overwrite:

Old data → deleted

New data → replaces it

Append:

Old data stays

New rows added



Microsoft Fabric

Step 3 – Historical Silver Layer

Run



```
1  from pyspark.sql.functions import col
2
3  # Read Bronze History
4  df = spark.table("bronze_weather_history")
5
6  # Flatten JSON + retain timestamp
7  silver_df = df.select(
8      col("name").alias("city"),
9      col("main.temp").alias("temperature"),
10     col("main.humidity").alias("humidity"),
11     col("weather")[0]["main"].alias("weather_condition"),
12     col("wind.speed").alias("wind_speed"),
13     col("sys.country").alias("country"),
14     col("ingestion_time")
15 )
16
17 # Show data
18 silver_df.show()
19
20 # Save Silver history
21 silver_df.write.format("delta") \
22     .mode("overwrite") \
23     .saveAsTable("silver_weather_history")
```

[8] ✓ 9 sec - Command executed in 9 sec 761 ms by Ahmed Fabric on 5/23/2026, 2:35:43 PM

> Spark jobs (13 of 13 succeeded) Resources Log



silver_weather_history

Showing 1 row

Table view

	ABC city	12 temperature	12L humidity	ABC weather_condition	12 wind_speed	ABC country	ingestion_time
1	London	24.13	57	Clouds	2.57	GB	2026-05-23T09:31:04.839Z

Microsoft Fabric

Step 4 – Historical Gold Layer

Now, creating business ready historical data

```
> | v
1  from pyspark.sql.functions import when, col
2
3  # Read Silver History
4  df = spark.table("silver_weather_history")
5
6  # Create Gold Layer
7  gold_df = df.withColumn(
8      "temp_category",
9      when(col("temperature") < 10, "Cold")
10     .when(col("temperature") < 20, "Moderate")
11     .otherwise("Hot")
12 )
13
14 # Show data
15 gold_df.show()
16
17 # Save Gold History
18 gold_df.write.format("delta") \
19     .mode("overwrite") \
20     .saveAsTable("gold_weather_history")
[9] ✓ 10 sec - Command executed in 9 sec 915 ms by Ahmed Fabric on 5/23/2026, 2:39:19 PM
> ⚙️ Spark jobs (13 of 13 succeeded) 📄 Resources
```

gold_weather_history

Showing 1 row

Table view ▾

	ABC city	12 temperature	12L humidity	ABC weather_condition	12 wind_speed	ABC country	🕒 ingestion_time	ABC temp_category
1	London	24.13	57	Clouds	2.57	GB	2026-05-23T09:31:04.839Z	Hot

After this, my architecture becomes,

OpenWeather API



bronze_weather_history



silver_weather_history



gold_weather_history

Now each rerun adds a new weather snapshot.

After that, I'll do the next big jump:

- Partitioning by date
- Scheduling automation
- Power BI dashboard using live historical trends
- Incremental API engineering with watermarks

Right now, my gold_weather_history stores all records together.

As history grows:

London → hourly data

1 day → 24 rows

1 month → 720 rows

1 year → 8,760+ rows

multiple cities → much larger

Scanning everything becomes inefficient.

GOAL:

Partition by:

- Year
- month

So, Spark reads only required data.

Example:

Current:

gold_weather_history

└─ all data together

After partitioning:

gold_weather_partitioned

|

└─ year = 2026

| └─ month = 5

| └─ month = 6

└─ year = 2027

Microsoft Fabric

Partition For API Historical

Data

Step 5 – Historical Gold Layer

Now, creating business ready historical data

```
1 from pyspark.sql.functions import year, month, col
2
3 # Read Gold History
4 df = spark.table("gold_weather_history")
5
6 # Create partition columns
7 partitioned_df = df.withColumn(
8     "year",
9     year(col("ingestion_time"))
10 ).withColumn(
11     "month",
12     month(col("ingestion_time"))
13 )
14
15 # Show data
16 partitioned_df.show()
17
18 # Save partitioned table
19 partitioned_df.write.format("delta") \
20     .mode("overwrite") \
21     .partitionBy("year", "month") \
22     .saveAsTable("gold_weather_partitioned")
```

[11] ✓ 12 sec - Command executed in 11 sec 980 ms by Ahmed Fabric on 5/23/2026, 3:03:50 PM

> ⚙ Spark jobs (14 of 14 succeeded) 📄 Resources 📄 Log

What this does?

Adds,

city	temperature	ingestion_time	year	month
London	12.1	2026-05-15	2026	5

Then physically stores data in partitions.

gold_weather_partitioned Showing 1 row

Table view ▾

	ABC city	12 temperature	12L humidity	ABC weather_condi...	12 wind_speed	ABC country	🕒 ingestion_time	ABC temp_category	123 year	123 month
1	London	24.13	57	Clouds	2.57	GB	2026-05-23T09:31:...	Hot	2026	5

Microsoft Fabric

AUTOMATION (PIPELINE SCHEDULING)

Step 6 – AUTOMATION

The screenshot shows the Microsoft Fabric interface. At the top, the user is logged in as 'Fabric Ahmed' with the project name 'Ahmed End-To-End Fabric Project'. A red box highlights the '+ New item' button in the top navigation bar. Below this, a 'Pipeline' card is shown with the description 'Ingest data at scale and schedule data workflows.' and a star icon. A red arrow points from the 'Pipeline' card to the 'New Pipeline' form. The form has fields for 'Name' (containing 'Weather_API_Pipeline') and 'Location' (set to 'Fabric Ahmed'). A red box highlights the 'Create' button. Below the form, a red arrow points to the 'Build a pipeline to organize and...' screen. This screen has two main sections: 'Start with a blank canvas' and 'Start with guidance'. The 'Start with a blank canvas' section shows a 'Pipeline activity' card with the description 'Automate data orchestrations using rich no-code activities.' and a red box around it. The 'Start with guidance' section shows a list of templates, with 'Notebook' highlighted by a red box and a red arrow pointing to it.

Step 1:

- I added New item, inside the workspace.
- And selected Pipeline, then created!
- After that, selected Notebook as Pipeline Activity.

Microsoft Fabric

AUTOMATION (PIPELINE SCHEDULING)

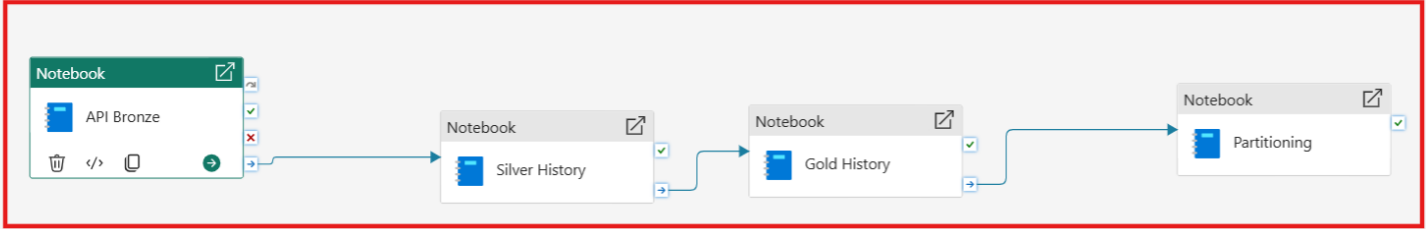
Step 6 – AUTOMATION

**Ahmed Irshad Hussain**
sardarahmed.irshad2@gmail.com
© COPYRIGHT

Fabric Ahmed_API_Notebook Ahmed_Sales_Lakehouse Weather_API_Pipeline

Home Activities Run View

Validate Run Schedule Trigger View run history Copy data Dataflow Notebook Lookup



General Settings

Please review this item carefully before adding it to the pipeline, as others in your organization may have access to notebooks in this workspace.

Connection Select... Refresh

Workspace * Fabric Ahmed Refresh

Notebook * Ahmed_API_Notebook Refresh Open New

> Base parameters

< Advanced settings

Session tag

Step 2:

- Inside Pipeline, I added four activities and assigned them the Notebook.
- After that, I connected every activity. (Look at the arrows), it means first bronze will run then silver then gold and then partition.
- After that, select Notebook as Pipeline Activity.

Step 3:

- I clicked the Run, to check the status of the pipeline.
- As all succeeded, it means there's no error in my pipeline.

Fabric Ahmed_API_Notebook Weather_API_Pipeline Ahmed_Sales_Lakehouse Search

Home Activities Run View

Validate Run Schedule Trigger View run history Copy data Dataflow Notebook Lookup Invoke Pipeline

Notebook API Bronze Silver History Gold History Partitioning

Parameters Variables Settings **Output** Library variables

Pipeline run ID 739a384a-bb80-487f-9669-3feb088574d5 Pipeline status **Succeeded** Export to CSV Filter Column options

Filter by keyword Showing 1 - 4 items

Activity name	Activity status	Run start	Duration	Input	Output
Partitioning	✓ Succeeded	5/24/2026, 2:25:53 PM	2m 10s	→	→
Gold History	✓ Succeeded	5/24/2026, 2:23:44 PM	2m 8s	→	→
Silver History	✓ Succeeded	5/24/2026, 2:21:36 PM	2m 7s	→	→
API Bronze	✓ Succeeded	5/24/2026, 2:19:12 PM	2m 22s	→	→

Microsoft Fabric

AUTOMATION (PIPELINE SCHEDULING)

Step 6 – AUTOMATION

Why I set hourly?

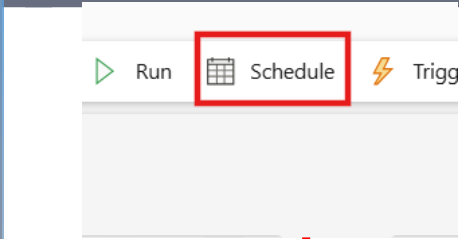
Because weather changes over time:

5:00 PM → 12°C

6:00 PM → 13°C

7:00 PM → 11°C

Now history grows automatically.



Schedules

+ Add schedule

Add schedule

Schedule type *

☒ Fixed
Run this schedule at fixed times on the clock or calendar.

☐ Interval-based
Wait a specific amount of time between each update.

Repeat * **Interval ***

Hourly 1 hours

Start date and time * **End date and time ***

05/24/2026 02:31 PM 05/24/2027 02:31 PM

Time zone *

(UTC+05:00) Islamabad, Karachi

Save Cancel

Schedules

Every hour

Last successful update **Next update**

24/05/2026, 14:32 49 minute(s)

Start **End** **Time zone**

24/05/2026, 14:31 24/05/2027, 14:31 (UTC+05:00) Islamabad, Karachi

Job type **Modified By**

Pipeline Ahmed Fabric

ID: 6f30c0c0-de28-429c-89e3-5373545d1b2b

Run Edit

Step 4 – Add Schedule:

- Click schedule.
- As I used Weather API, so it means I need data every hour.
- I set the schedule hourly based on Pakistani Time!
- As you can see in the third screen shot, Last update was on 2:32 pm (PKT) and the next update will be by 3:32 pm (PKT).

Microsoft Fabric

POWER BI DASHBOARD

Step 7 – POWER BI

I'll use,

- 4 KPI Cards for City, Current Temperature, humidity, Wind Speed &
- a Line Chart for Wind Speed by Ingestion Time)

**Ahmed Irshad Hussain**
sardarahmed.irshad2@gmail.com
© COPYRIGHT

New semantic model

Direct Lake semantic model name ⓘ

Weather Dashboard

Workspace ⓘ

Fabric Ahmed

Select or deselect tables for the semantic model.

Search ⓘ

SQL views cannot be selected for Direct Lake on OneLake, but tables based on SQL views can be added to the semantic model later using other storage modes.

☒ Select all

☐ gold_weather_history

☒ gold_weather_partitioned

☐ silver_sales

☐ silver_weather

☐ silver_weather_history

[Learn more about Direct Lake](#) ⓘ

Please wait...

Confirm

Click new Semantic Model,

And then choose the table on which you want to create your dashboard!

City

London

Current Temperature

Sunny

30.95

Current Temperature KPI

Wind Speed

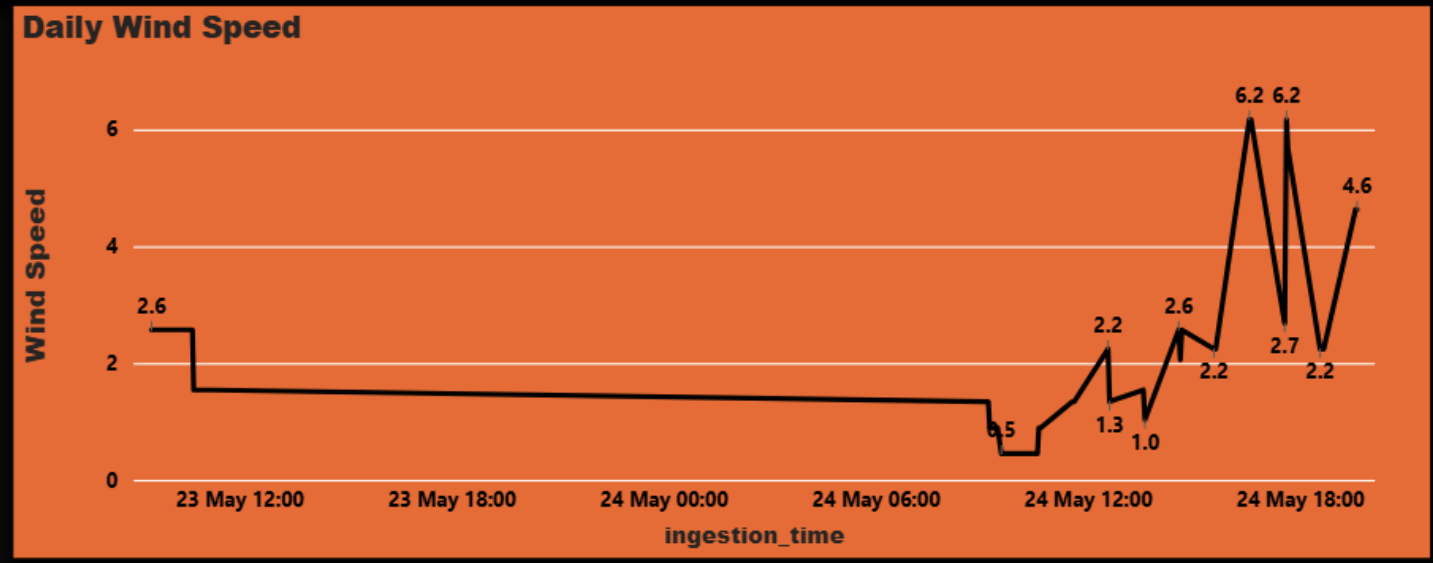
6.17

Max Daily Wind Speed

Current Humidity

62

Current Humidity KPI



Weather API Analytics – End to End Flow

